

A Linear-Time Algorithm for the Positive Two-Edge-Cut Kernelization Rule

opinion_projection / signed_graph_kernelization

2026

Overview

This note explains, from first principles, the algorithm used by kernelization **rule (iv)** (*positive edge separator*) in `signed_graph_kernelization.py`, in particular how it finds a cut of **one or two positive edges** in $O(|V| + |E|)$ time.

The hard part is the **two-edge** case: deciding whether there exist two edges whose removal disconnects the graph, and doing so without trying all $\binom{|E|}{2}$ pairs. The trick is to compute, for every edge, a small **signature** such that

two edges form a 2-edge cut **iff** they have the same (non-zero) signature.

All signatures are computed in a single depth-first search. The remainder of this document derives why that is true and how the signatures are computed.

We assume throughout an **undirected, simple, connected** graph $G = (V, E)$. In the kernelizer this is the *combined* graph $G^+ \cup G^-$ of a signed graph; “positive” edges are those in G^+ . Rule (iv) is run after rules (i)–(iii), so G is already 2-vertex-connected (hence 2-edge-connected, i.e. bridgeless) — but the algorithm below is stated for a general connected graph and detects bridges as a degenerate case.

1. What we are looking for

A **k -edge cut** is a set of k edges whose removal increases the number of connected components.

- A **bridge** is a 1-edge cut.
- A **2-edge cut** is a pair $\{e, f\}$ such that $G - \{e, f\}$ is disconnected but $G - e$ and $G - f$ are both connected.

Rule (iv) wants a 1- or 2-edge cut **all of whose edges are positive**, because such a cut lets us solve the two sides of the graph independently and then glue the solutions back together along the (few) positive edges without introducing any projection error.

The 1-edge case is easy: a positive bridge. Bridges are found in $O(|V| + |E|)$ by the textbook DFS-lowpoint algorithm (we just call `networkx.bridges`). The rest of this note is about the 2-edge case.

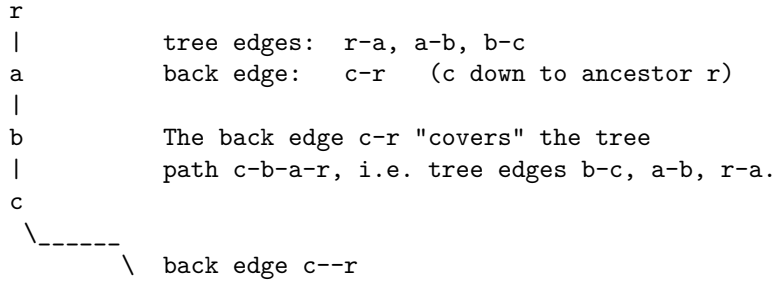
2. DFS trees, tree edges, and back edges

Run a depth-first search from an arbitrary root. Every vertex gets a **discovery time** `disc[v]` (the order in which DFS first reaches it). The DFS classifies each edge into one of two kinds:

- **Tree edge** — an edge used to first reach a new vertex. The tree edges form a spanning tree T .
- **Back edge** — every *other* edge.

A crucial fact about DFS on **undirected** graphs: there are **no cross edges**. Every non-tree edge connects a vertex to one of its **ancestors** in T . (There is simply no way, in undirected DFS, to have an edge between two vertices where neither is an ancestor of the other — one of them would have discovered the other first.)

So we can speak unambiguously of a back edge $b = (u, v)$ with v an ancestor of u .



Fundamental cycles

Each back edge $b = (u, v)$ defines exactly one **fundamental cycle** C_b : the back edge b together with the unique tree path from u up to v . We say b **covers** a tree edge t if t lies on that tree path. Two properties of C_b are used repeatedly below, so we name them now:

- **(F1)** C_b is a *simple* cycle of G (a back edge plus a simple tree path).
- **(F2)** C_b is the **only** fundamental cycle that contains the back edge b . Indeed, apart from b itself, C_b consists entirely of *tree* edges, so no *other* $C_{b'}$ (which contains only the back edge b') can contain b .

These fundamental cycles are a basis of the cycle space of G ; there are exactly $|E| - |V| + 1$ of them (one per back edge). Everything below is really a statement about this cycle space, made computational.

3. The key theorem: 2-edge cuts = “cycle-equivalent” edges

Definition (cycle equivalence). Two edges e and f are *cycle-equivalent* if **every** cycle of G that contains e also contains f , and vice versa.

Theorem. In a 2-edge-connected graph, $\{e, f\}$ is a 2-edge cut **iff** e and f are cycle-equivalent.

Proof of the “cut $\Rightarrow$ equivalent” direction. Suppose $\{e, f\}$ is a 2-edge cut. Removing both splits V into two parts S and $V \setminus S$, and e, f are the only two edges crossing between them. Now take any cycle C . A cycle crosses any vertex partition an **even** number of times. The only crossing edges available are e and f , so C crosses either **0** times (it stays entirely on one side and contains neither e nor f) or **2** times (it contains both e and f). Hence every cycle contains e iff it contains f — they are cycle-equivalent. ■

The “*equivalent $\Rightarrow$ cut*” direction is also true (it is the basis of the linear-time 3-edge-connectivity algorithms) but a touch more involved; the clean way to see it is the **cactus picture** in §6. Our implementation does not rely on this direction being perfect: it **verifies** every candidate cut by actually removing the two edges and checking connectivity (§5), so a wrong candidate can never produce a wrong split.

Why this helps. Cycle equivalence is an *equivalence relation*, so it partitions the edges into classes. By the theorem, **any two edges in the same class form a 2-edge cut**. So instead of testing pairs, we just need to **bucket edges by their class** and look for a bucket containing ≥ 2 positive edges.

4. Turning cycle classes into computable signatures

We now make “which class is this edge in” computable. The idea: characterize an edge by the **set of fundamental cycles it belongs to**. Write

$$\Phi(e) = \{b : e \in C_b\}$$

for that set. By the definition of “covers,” for a tree edge this is $\Phi(t) = \text{cov}(t) = \{b : b \text{ covers } t\}$, and by **(F2)** a back edge sits in exactly one fundamental cycle — its own — so $\Phi(b) = \{b\}$. Note $\Phi(e)$ is always a set of *back edges*.

The entire method rests on the following equivalence.

Lemma. e and f are cycle-equivalent **iff** $\Phi(e) = \Phi(f)$.

Proof, direction $\Phi(e) \neq \Phi(f) \Rightarrow$ not equivalent. Pick a back edge b in the symmetric difference $\Phi(e) \triangle \Phi(f)$, say $e \in C_b$ but $f \notin C_b$. By **(F1)**, C_b is a *simple* cycle, and it contains e but not f . That single cycle witnesses that e and f are not cycle-equivalent.

Proof, direction $\Phi(e) = \Phi(f) \Rightarrow$ equivalent. This needs only two concrete facts; no cycle-space machinery.

Fact 1 (a cycle crosses any split evenly). Split the vertices into two groups A and B , and call an edge a *crossing edge* if its two endpoints fall in different groups. Then every cycle uses an **even** number of crossing edges. *Why:* walk once around the cycle and back to the start; each step from A into B must eventually be matched by a step from B back into A , so the side-changing steps pair up.

Fact 2 (a tree edge is the lone tree edge of its own cut). Fix a tree edge $t = (p, c)$ and let D be the set of vertices in the subtree below c . Removing t from the spanning tree separates exactly D from the rest, so t is the **only tree edge** crossing the split $(D, V \setminus D)$. Every other crossing edge is thus a back edge — and a back edge crosses $(D, V \setminus D)$ exactly when one endpoint is inside the subtree and the other is outside, which is precisely the condition that t lies on that back edge’s tree path, i.e. that the back edge *covers* t . Therefore

$$\{\text{edges crossing } (D, V \setminus D)\} = \{t\} \cup \Phi(t).$$

Now combine the two. Take any simple cycle C and any **tree** edge t . By Fact 1 (with the split $(D, V \setminus D)$) the number of crossing edges that C uses is even, and by Fact 2 those crossing edges are t together with the back edges in $\Phi(t)$. Since t is a tree edge and the members of $\Phi(t)$ are back edges, these are distinct, so

$$\underbrace{[t \in C]}_{0 \text{ or } 1} + |C \cap \Phi(t)| \equiv 0 \pmod{2}, \quad \text{i.e.} \quad [t \in C] \equiv |C \cap \Phi(t)| \pmod{2}.$$

Here $C \cap \Phi(t)$ is the set of back edges of C that cover t . For a **back** edge b the same identity holds trivially, because $\Phi(b) = \{b\}$ makes the right-hand side just $[b \in C]$. So for **every** edge e and every simple cycle C ,

$$[e \in C] \equiv |C \cap \Phi(e)| \pmod{2}. \quad (\star)$$

Finally, suppose $\Phi(e) = \Phi(f)$. Then for any simple cycle C the right-hand side of $(\star)$ is the same number for e and for f , so $[e \in C]$ and $[f \in C]$ have equal parity; being each 0 or 1, they are simply equal. Hence $e \in C \iff f \in C$ for **every** cycle C — the two edges are cycle-equivalent. ■

The lemma is what lets a per-edge label stand in for a whole equivalence class: the **signature** of an edge encodes *its set of fundamental cycles* $\Phi(e)$. We do this with randomization:

Give each back edge b an independent uniformly random label $L_b \in \{0, 1\}^{64}$. Define the signature of an edge as the bitwise **XOR** of the labels of the fundamental cycles it belongs to.

Concretely:

$$\text{sig}(t) = \bigoplus_{b \text{ covers } t} L_b \quad (\text{tree edge } t), \quad \text{sig}(b) = L_b \quad (\text{back edge } b).$$

Why XOR of random labels identifies the set. If two edges belong to the same set S of cycles, their signatures are both $\bigoplus_{b \in S} L_b$ — equal, always. If they belong to **different** sets $S \neq S'$, their signatures differ unless $\bigoplus_{b \in S} L_b = \bigoplus_{b \in S'} L_b$, i.e. $\bigoplus_{b \in S \Delta S'} L_b = 0$ for a non-empty symmetric difference $S \Delta S'$. For independent uniform 64-bit labels this happens with probability exactly 2^{-64} per pair. With m edges there are fewer than $m^2/2$ pairs, so the chance of **any** false coincidence is below $m^2/2^{65}$ — about 10^{-7} even for a graph with two million edges. And we verify before acting, so even that is caught (§5).

Two special values fall out for free:

- $\text{sig}(t) = 0 \iff \text{cov}(t) = \emptyset \iff t$ is a **bridge** (covered by no back edge). (Probabilistically; an empty XOR is 0 exactly, and a non-empty XOR is non-zero w.h.p.)
- Two edges with **equal non-zero** signature are (w.h.p.) cycle-equivalent, and hence form a **2-edge cut**.

5. Computing all signatures in one DFS (the subtree-XOR trick)

The only non-obvious part left is computing $\text{sig}(t)$ for **every** tree edge t efficiently. Naively, a single back edge can cover $\Theta(|V|)$ tree edges, so writing its label onto each covered edge is $O(|V| \cdot |E|)$. We avoid that with a classic **difference / subtree-aggregation** trick.

For each tree edge, identify it with its lower endpoint: the tree edge above vertex c is $(\text{parent}(c), c)$. Let $\text{subtree}(c)$ be the set of vertices in the DFS subtree rooted at c . The key observation:

A back edge $b = (x, y)$ covers the tree edge above c **iff exactly one** of x, y lies in $\text{subtree}(c)$.

(If both endpoints are inside the subtree, the cycle stays within it and does not cross the edge above c ; if both are outside, likewise; the path crosses the boundary edge exactly when one endpoint is in and one is out.)

So define a per-vertex accumulator

$$\text{delta}[v] = \bigoplus_{\substack{b=(x,y) \\ x=v \text{ or } y=v}} L_b$$

— each back edge XORs its label into **both** of its endpoints. Then aggregate over subtrees:

$$\text{subxor}[c] = \bigoplus_{v \in \text{subtree}(c)} \text{delta}[v].$$

Claim: $\text{subxor}[c] = \text{sig}((\text{parent}(c), c))$.

Proof. Expand $\text{subxor}[c]$ by back edge. A back edge $b = (x, y)$ contributes L_b once for each of its endpoints lying in $\text{subtree}(c)$:

- both endpoints inside $\Rightarrow L_b$ XORed twice $\Rightarrow$ cancels to 0;
- neither inside $\Rightarrow$ contributes 0;
- exactly one inside $\Rightarrow$ contributes L_b once.

By the observation above, “exactly one inside” is precisely “ b covers the edge above c .” Hence $\text{subxor}[c] = \bigoplus_{b \text{ covers } t} L_b = \text{sig}(t)$. ■

The subtree XOR is computed bottom-up: process vertices in **reverse** discovery order (children before parents), recording each tree edge’s signature and then folding the child’s accumulated value into its parent:

```
subxor = copy(delta)
for node in reversed(discovery_order):
    p = parent[node]
    if p exists:
        sig[(p, node)] = subxor[node]      # signature of the tree edge above `node`
```

```
subxor[p] ^= subxor[node]           # fold subtree into the parent
```

Back-edge signatures are recorded directly as $\text{sig}(b) = L_b$ at the moment the back edge is discovered.

Iterative DFS detail

For large graphs we cannot recurse (Python's stack would overflow at a few thousand levels). The DFS is therefore an explicit stack, and — importantly — each stack frame **keeps its neighbour iterator**. When we pause a vertex to descend into a freshly discovered child and later resume it, the remaining neighbours are examined *after* those vertices have been visited, so they are correctly recognized as back edges rather than re-used as tree edges. Each undirected edge is examined from both endpoints; we count a back edge exactly once, from the deeper endpoint, using the test `w != parent and disc[w] < disc[u]`.

From signatures to a positive cut

1. (1-edge) If any bridge is a positive edge, return it as a 1-edge cut.
2. (2-edge) Compute $\text{sig}(e)$ for every edge in one DFS.
3. Group the positive edges by signature, skipping signature 0 (bridges).
4. The first group containing ≥ 2 positive edges gives a candidate $\{e_1, e_2\}$.
5. VERIFY: remove e_1, e_2 and check the component count increased. If yes, return the cut; otherwise (a hash collision - essentially never) keep looking.

Step 5 makes the procedure **Las Vegas**: the randomness only ever affects *whether we find* a cut, never whether a returned cut is *correct*.

6. Worked examples

Example A — a 4-cycle (every pair of edges is a 2-cut)

Vertices 1–2–3–4–1. DFS from 1 gives tree edges 1-2, 2-3, 3-4 and a single back edge 4-1 with label L .

- delta: L at vertex 4 and at vertex 1, 0 elsewhere.
- $\text{subxor}[4] = L$, $\text{subxor}[3] = 0 \oplus L = L$, $\text{subxor}[2] = 0 \oplus L = L$.
- Signatures: 1-2 $\mapsto L$, 2-3 $\mapsto L$, 3-4 $\mapsto L$, back edge 4-1 $\mapsto L$.

All four edges share signature L : one class. Indeed, in a cycle **any** two edges form a 2-edge cut. ✓

Example B — two triangles sharing an edge

Triangles 1-2-3 and 1-2-4 sharing edge 1-2. Edges: 1-2, 1-3, 2-3, 1-4, 2-4. DFS from 1: tree edges 1-2, 2-3, 2-4; back edges 3-1 (label L_1) and 4-1 (label L_2).

- delta: vertex 3 $\mapsto L_1$, vertex 4 $\mapsto L_2$, vertex 1 $\mapsto L_1 \oplus L_2$, vertex 2 $\mapsto 0$.
- $\text{subxor}[3] = L_1$, $\text{subxor}[4] = L_2$, $\text{subxor}[2] = 0 \oplus L_1 \oplus L_2 = L_1 \oplus L_2$.
- Signatures: 2-3 $\mapsto L_1$, back 3-1 $\mapsto L_1 \rightarrow$ **class** $\{2-3, 3-1\}$ (cut isolating vertex 3); 2-4 $\mapsto L_2$, back 4-1 $\mapsto L_2 \rightarrow$ **class** $\{2-4, 4-1\}$ (cut isolating vertex 4); 1-2 $\mapsto L_1 \oplus L_2 \rightarrow$ **alone**, in no 2-edge cut.

This matches reality: removing $\{2-3, 1-3\}$ isolates vertex 3, removing $\{2-4, 1-4\}$ isolates vertex 4, but the shared edge 1-2 is in no 2-edge cut. ✓

7. Complexity

Step	Cost
DFS + back-edge labelling	$O(V + E)$
Subtree-XOR aggregation	$O(V)$
Grouping edges by signature	$O(E)$
One verification (BFS)	$O(V + E)$
Total per call	$O(V + E)$

This replaces the previous brute-force two-edge search, which removed each positive edge and recomputed bridges — $O(|E^+| \cdot (|V| + |E|))$ — and was only affordable below a size threshold.

8. Mapping to the code

In `graph_utils/signed_graph_kernelization.py`:

- `_edge_signatures(combined)` — the single DFS of §5. Uses `random.Random(0)` (fixed seed → deterministic, reproducible runs), an explicit DFS stack that keeps each vertex’s neighbour iterator, the `delta` accumulator, and the reverse-discovery-order subtree-XOR pass. Returns a dict `frozenset({u, v}) -> signature`.
 - `_find_positive_edge_cut(combined, positive)` — §4 step 1 (positive bridge via `nx.bridges`) followed by §5’s grouping of positive edges by signature and the verification step.
 - `_disconnects(combined, edges)` — the verification: remove the edges on a copy and compare connected-component counts.
 - `split_at_positive_edge_separator(graph)` — wraps the above: builds the combined graph, finds a cut, removes it, and returns the induced signed subgraphs of the resulting components (the cut’s positive edges are dropped, to be matched later when the sub-solutions are glued together).
-

9. The remaining inefficiency (and the proper fix)

The *per-call* cost is now linear, but the **driver** `kernelise_graph` re-applies all rules after every split. A single positive 2-edge cut usually peels off only a tiny fragment, after which the large remaining component is re-scanned from scratch. With dozens of such cuts on a large graph this means dozens of full $O(|V| + |E|)$ passes — empirically a 10–20× slowdown on the biggest datasets for a sub-0.5% additional reduction.

The principled fix is to find **all** 2-edge cuts in one pass via the **cactus / 3-edge-connected-component** structure:

1. The signature classes *are* the cycle-equivalence classes. Contract every edge whose class has size 1 (these lie inside a 3-edge-connected component and are in no 2-edge cut). The quotient graph is a **cactus** — a connected graph in which every edge lies on exactly one simple cycle.
2. In a cactus, the 2-edge cuts are exactly the pairs of edges on a common cycle. One traversal enumerates them all.
3. For rule (iv) we additionally require the cut edges to be **positive** and must ensure no resulting component boundary accumulates more than two crossing positive edges (otherwise the affine gluing step is no longer valid). That extra bookkeeping is the only subtlety.

This turns full rule (iv) into a single near-linear decomposition rather than a sequence of re-scans. It is left as future work because, on the datasets measured, rule (iv) contributes very little beyond rules (i)–(iii).

10. Further reading (optional)

The construction above is self-contained; these are pointers only, not load-bearing citations.

- **Cycle equivalence of edges, linear time.** R. Johnson, D. Pearson, K. Pingali, *The Program Structure Tree: Computing Control Regions in Linear Time*, PLDI 1994. (The deterministic version of §3–§4: edges are grouped by the set of “brackets”/back edges covering them.)
- **Bridges & 2-edge-connectivity, linear time, very clean.** J. M. Schmidt, *A Simple Test on 2-Vertex- and 2-Edge-Connectivity*, Information Processing Letters, 2013 (chain decomposition).
- **3-edge-connected components / all 2-cuts.** Y. H. Tsin, *Yet another optimal algorithm for 3-edge-connectivity*, J. Discrete Algorithms, 2009; Mehlhorn, Neumann, Schmidt, *Certifying 3-Edge-Connectivity*, Algorithmica, 2017. (Relevant to the batched fix in §9.)

The random-label-XOR shortcut itself (§4–§5) is a folklore combination of two standard ideas — XOR hashing for set-equality, and the subtree-difference trick for path aggregation — rather than a single citable result.